Algoritmi ricorsivi (2/2)

Sapienza Università di Roma

Indice

Ricerca binaria ricorsiva

Il problema del sotto-array di somma massima

Riepilogo del capitolo

Dove eravamo

- Nella lezione precedente abbiamo studiato due algoritmi ricorsivi sugli array, con lo schema dell'analisi di ogni algoritmo ricorsivo: pseudocodice e albero delle chiamate, correttezza per induzione forte, equazione di ricorrenza (albero di ricorsione e Master theorem), tempo di esecuzione, memoria aggiuntiva
- **La somma di un array**: `RecursiveSum` esegue $7n - 5 = \Theta(n)$ passi, come l'algoritmo iterativo, ma usa $\Theta(\log_2 n)$ di memoria per la pila delle chiamate ($\Theta(n)$ per la versione per decremento di uno)
- **Merge Sort**: due chiamate ricorsive sulle metà e una `Merge` in $\Theta(n)$ passi; $S(n) = 2S(n/2) + \Theta(n) = \Theta(n \log_2 n)$, contro $\Theta(n^2)$ dell'Insertion Sort

Questa lezione

- La **ricerca** in un array ordinato, con la **ricerca binaria ricorsiva**: una sola chiamata ricorsiva su una metà
- Il problema del **sotto-array di somma massima**, in cui un algoritmo divide et impera migliora la forza bruta, e un algoritmo iterativo ingegnoso migliora entrambi
- Alla fine, un riepilogo dell'intero capitolo

Ricerca binaria ricorsiva

Introduzione: Ricerca binaria ricorsiva

- Il Merge Sort effettua due chiamate ricorsive sulle due metà. Il prossimo algoritmo ne effettua soltanto **una**: dopo un solo confronto, una delle due metà può essere scartata
- Scriviamo la ricerca binaria del capitolo «Ricerca lineare e binaria» come algoritmo ricorsivo, e la analizziamo con lo stesso schema
- Stesso tempo di esecuzione della versione iterativa, ma più memoria: questo ci porta alla **ricorsione in coda**

Il problema

Dato un array $a[1..n]$ di n numeri naturali
ordinati in ordine non decrescente e un numero naturale m da cercare,
consideriamo il problema di determinare
se m è uno degli elementi dell'array.

- Nel capitolo «Ricerca lineare e binaria» abbiamo risolto questo problema con l'algoritmo **iterativo** `BinarySearch`

L'algoritmo iterativo

- L'algoritmo mantiene un intervallo di ricerca $a[left..right]$, inizialmente $a[1..n]$
- A ogni iterazione, confronta il valore cercato m con l'elemento centrale

$$a[mid], \qquad mid = left + \left\lfloor \frac{right - left}{2} \right\rfloor .$$

- Se $a[mid] = m$ l'elemento è stato trovato; altrimenti, poiché l'array è ordinato, metà dell'intervallo può essere scartata

Lo stesso problema su un sotto-array più piccolo

- L'osservazione chiave è che, dopo il confronto con $a[mid]$, il problema che resta da risolvere è **lo stesso problema** (cercare m in un array ordinato) su un sotto-array **più piccolo**:
  - se $a[mid] < m$, allora m può trovarsi solo in $a[mid + 1..right]$;
  - se $a[mid] > m$, allora m può trovarsi solo in $a[left..mid - 1]$.

- Questa è esattamente la struttura richiesta da un algoritmo ricorsivo: risolviamo il problema su $a[left..right]$ risolvendo lo stesso problema su un sotto-array di dimensione (circa) metà

Casi base e parametri

- I **casi base** sono
  - l'intervallo vuoto ($left > right$), in cui m certamente non è presente
  - il caso $a[mid] = m$, in cui m viene trovato

- A differenza della versione iterativa, i limiti *left* e *right* sono ora **parametri** della funzione

- Per cercare m nell'intero array, chiamiamo

$$\texttt{RecursiveBinarySearch}(a, 1, n, m).$$

- Nelle prossime slide, lo pseudocodice dell'algoritmo di ricerca binaria ricorsiva

Algorithm: RecursiveBinarySearch (1/2)

Input : Un array $a[1..n]$ di n numeri naturali ordinati in ordine non decrescente, due indici *left* e *right*, e un valore da cercare m

Output : Un indice $i \in \{left, \ldots, right\}$ tale che $a[i] = m$, oppure -1 se m non è in $a[left..right]$

```
1 function RecursiveBinarySearch(a, left, right, m)
2     if left > right then
3         return −1   // Intervallo vuoto: non trovato
4     end
5     mid := left + ⌊(right − left)/2⌋     // Indice centrale
6     if a[mid] = m then
7         return mid                       // Elemento trovato
8     end
```

Algorithm: RecursiveBinarySearch (2/2)

```
function RecursiveBinarySearch(a, left, right, m) (continua)
    if a[mid] < m then
        return RecursiveBinarySearch(a, mid + 1, right, m)
    end
    return RecursiveBinarySearch(a, left, mid − 1, m)
end
```

Esegui in Colab: Python · C++ · Java

Come funziona `RecursiveBinarySearch` (1/2)

- Righe 2–3: se l'intervallo è vuoto ($left > right$), l'elemento non è presente e viene restituito -1 (**caso base**)
- Riga 5: l'indice centrale *mid* viene calcolato con la stessa formula dell'algoritmo iterativo
- Righe 6–7: se $a[mid] = m$, l'elemento è stato trovato e il suo indice viene restituito (**caso base**)

Come funziona `RecursiveBinarySearch` (2/2)

- Righe 9–10: se $a[mid] < m$, la ricerca prosegue **ricorsivamente** nella metà destra $a[mid + 1..right]$
- Riga 12: altrimenti $a[mid] > m$, e la ricerca prosegue **ricorsivamente** nella metà sinistra $a[left..mid - 1]$
- Il valore restituito dalla chiamata ricorsiva viene restituito invariato al chiamante

Ogni chiamata di `RecursiveBinarySearch` effettua **al più una** chiamata ricorsiva.
Quindi, l'**albero delle chiamate** è un **cammino**: non ha ramificazioni.

Questo è molto diverso dal Merge Sort, in cui ogni chiamata su un array
con almeno due elementi genera due chiamate ricorsive.

Esempi

- Consideriamo le stesse ricerche eseguite nel capitolo «Ricerca lineare e binaria» con l'algoritmo iterativo: cerchiamo quattro valori nell'array ordinato

$$a = [3, 9, 11, 12, 12, 16, 19] \qquad (n = 7)$$

- Per ogni ricerca riportiamo una riga per ogni chiamata: i valori di *left*, *right*, *mid*, $a[mid]$ e l'azione eseguita

- Poi disegniamo l'albero delle chiamate: una freccia nera è una chiamata ricorsiva, una freccia blu è il valore restituito al chiamante

Esempio (1/6)

Ricerca di $m = 12$. Prima chiamata `RecursiveBinarySearch`$(a, 1, 7, 12)$:

chiamata	*left*	*right*	*mid*	$a[mid]$	azione
1	1	7	4	12	$a[mid] = m \Rightarrow$ Trovato! Ritorna 4

`RecursiveBinarySearch`$(a, 1, 7, 12)$ $\longrightarrow$ restituisce 4

L'elemento 12 viene trovato all'indice 4 con una sola chiamata (nessuna ricorsione).

Esempio (2/6)

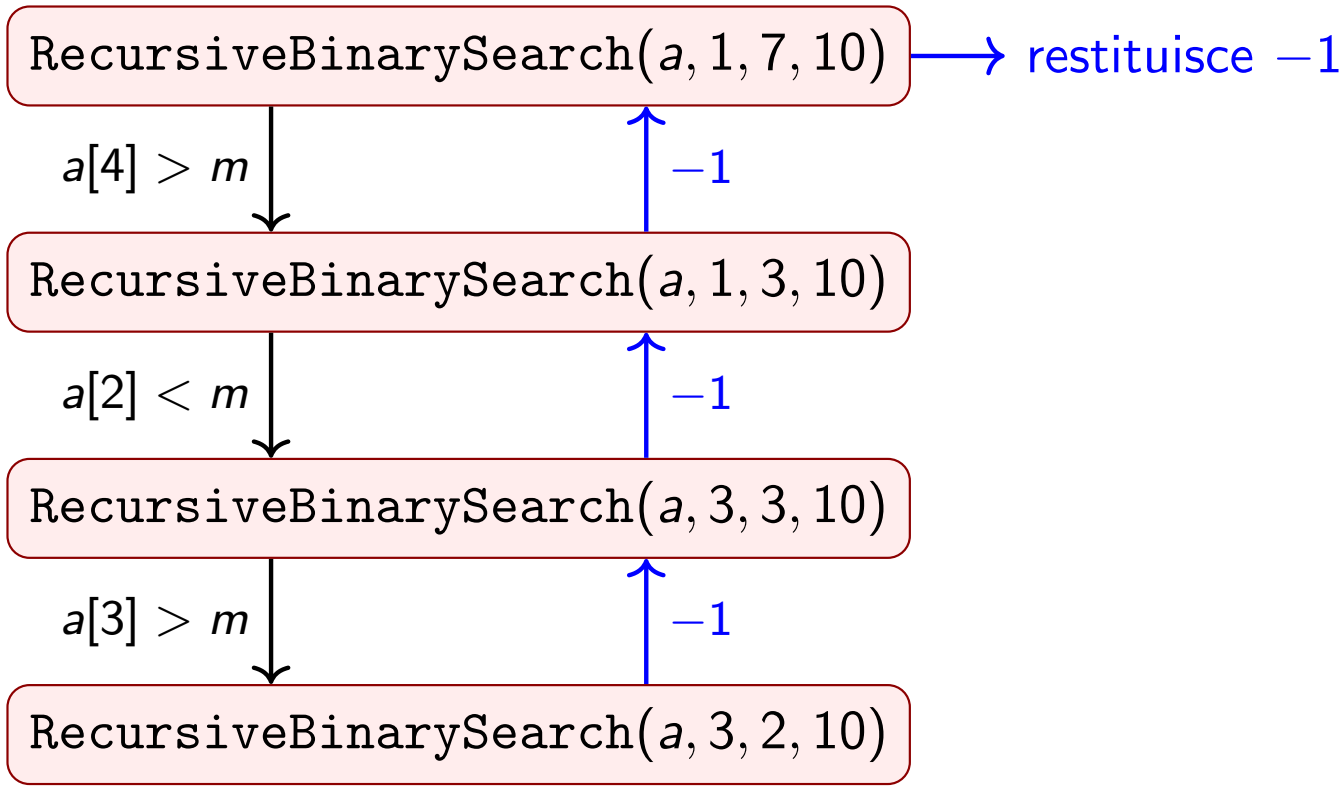
Ricerca di $m = 10$. Prima chiamata `RecursiveBinarySearch`$(a, 1, 7, 10)$:

chiamata	*left*	*right*	*mid*	$a[mid]$	azione
1	1	7	4	12	$a[mid] > m \Rightarrow$ chiama $left = 1$, $right = 3$
2	1	3	2	9	$a[mid] < m \Rightarrow$ chiama $left = 3$, $right = 3$
3	3	3	3	11	$a[mid] > m \Rightarrow$ chiama $left = 3$, $right = 2$
4	3	2	–	–	$left > right \Rightarrow$ Non trovato! Ritorna -1

L'elemento 10 non è nell'array: l'ultima chiamata è su un intervallo vuoto e restituisce -1, e questo valore viene propagato fino alla prima chiamata (albero delle chiamate nella prossima slide).

Esempio (3/6)

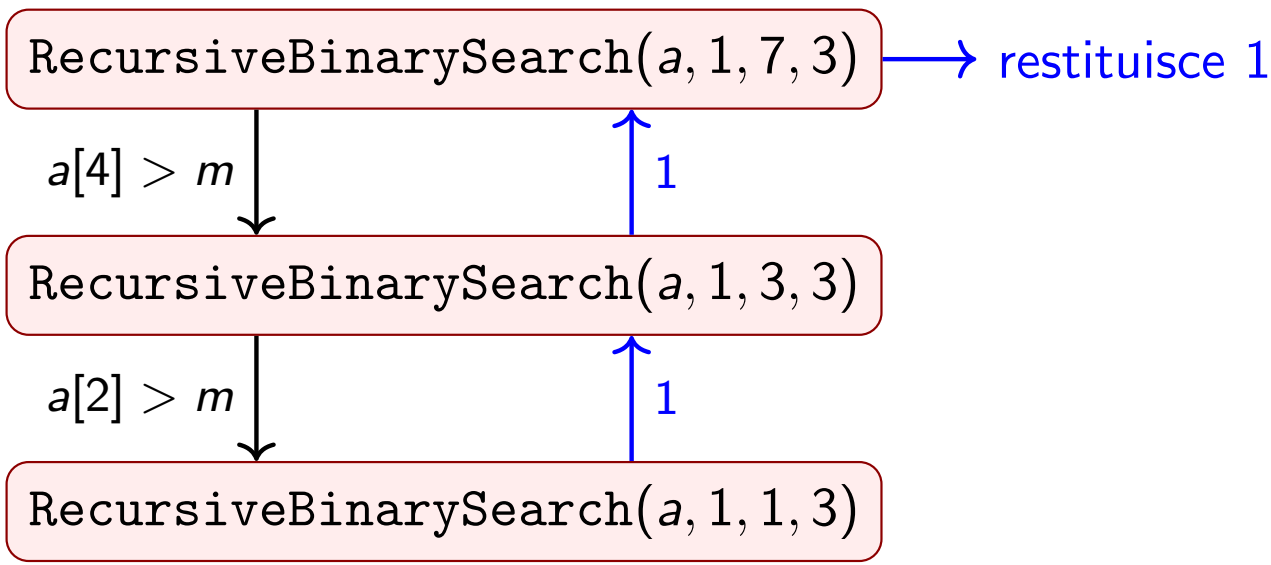
L'albero delle chiamate della ricerca di $m = 10$:



Esempio (4/6)

Ricerca di $m = 3$. Prima chiamata `RecursiveBinarySearch`$(a, 1, 7, 3)$:

chiamata	*left*	*right*	*mid*	$a[mid]$	azione
1	1	7	4	12	$a[mid] > m \Rightarrow$ chiama $left = 1$, $right = 3$
2	1	3	2	9	$a[mid] > m \Rightarrow$ chiama $left = 1$, $right = 1$
3	1	1	1	3	$a[mid] = m \Rightarrow$ Trovato! Ritorna 1

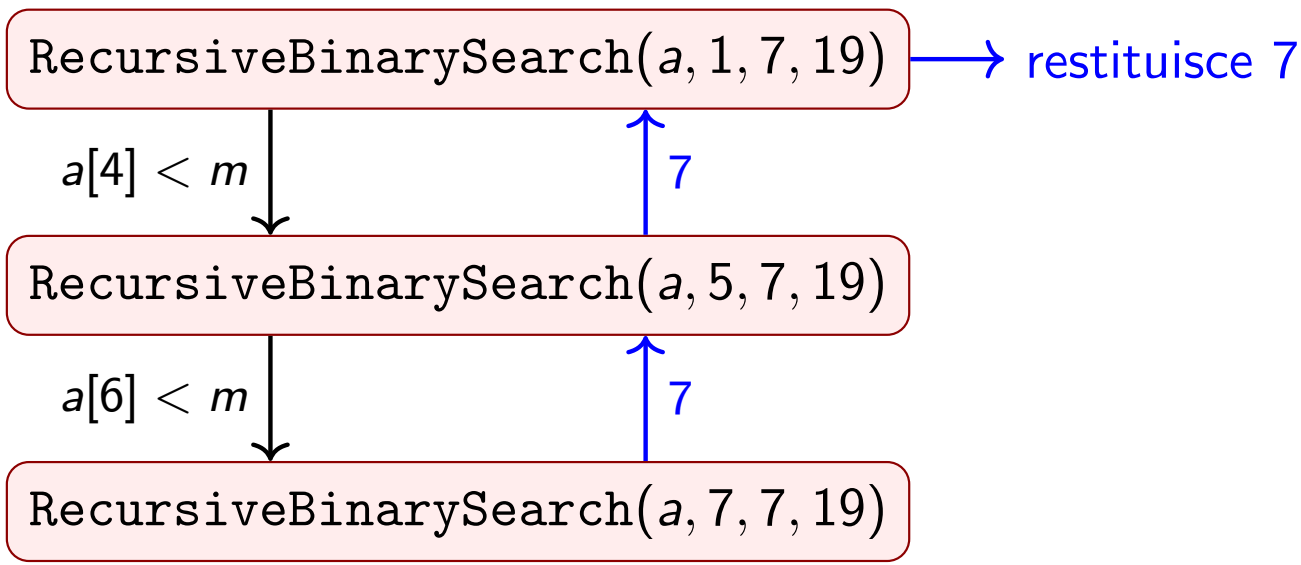


L'elemento 3 viene trovato all'indice 1.

Esempio (5/6)

Ricerca di $m = 19$. Prima chiamata `RecursiveBinarySearch`$(a, 1, 7, 19)$:

chiamata	*left*	*right*	*mid*	$a[mid]$	azione
1	1	7	4	12	$a[mid] < m \Rightarrow$ chiama $left = 5$, $right = 7$
2	5	7	6	16	$a[mid] < m \Rightarrow$ chiama $left = 7$, $right = 7$
3	7	7	7	19	$a[mid] = m \Rightarrow$ Trovato! Ritorna 7



L'elemento 19 viene trovato all'indice 7.

Esempio (6/6)

- La successione dei valori ($left$, $right$, mid) è **esattamente la stessa** prodotta dall'algoritmo iterativo
- Ogni chiamata dell'algoritmo ricorsivo corrisponde a un'iterazione del ciclo `while` dell'algoritmo iterativo
- Si noti che la seconda occorrenza di 12 (indice 5) non viene mai trovata cercando $m = 12$
- Come nella versione iterativa, l'algoritmo restituisce **un** indice i con $a[i] = m$, non necessariamente il primo

Correttezza

- Come per gli algoritmi precedenti, la correttezza si dimostra per **induzione forte** sulla dimensione dell'input della chiamata
- La dimensione della chiamata `RecursiveBinarySearch`$(a, \mathit{left}, \mathit{right}, m)$ è il numero di elementi dell'intervallo:

$$k = \mathit{right} - \mathit{left} + 1.$$

- Per un intervallo vuoto ($\mathit{left} = \mathit{right} + 1$) si ha $k = 0$

Teorema 1: correttezza della ricerca binaria ricorsiva

Sia $a[1..n]$ *ordinato in ordine non decrescente e siano* $1 \leq left \leq right + 1 \leq n + 1$. *Allora* `RecursiveBinarySearch`$(a, left, right, m)$ *termina e restituisce:*

- *un indice* $i \in \{left, \ldots, right\}$ *tale che* $a[i] = m$, *se* m *è uno degli elementi di* $a[left..right]$;
- -1, *altrimenti.*

Dimostrazione (1/3)

Procediamo per induzione forte su $k = \mathit{right} - \mathit{left} + 1 \geq 0$.

Caso base $k = 0$. Allora $\mathit{left} > \mathit{right}$, l'intervallo è vuoto e m non è uno dei suoi elementi. L'algoritmo esegue le righe 2–3 e restituisce -1, il che è corretto.

Passo induttivo. Sia $k \geq 1$ e supponiamo (ipotesi induttiva) che l'enunciato valga per ogni chiamata il cui intervallo ha dimensione $k' < k$. Poiché $\mathit{left} \leq \mathit{right}$, l'algoritmo calcola

$$\mathit{mid} = \mathit{left} + \left\lfloor \frac{\mathit{right} - \mathit{left}}{2} \right\rfloor, \qquad \mathit{left} \leq \mathit{mid} \leq \mathit{right}.$$

Dimostrazione (2/3)

- Se $a[mid] = m$, l'algoritmo restituisce $mid \in \{left, \ldots, right\}$, il che è corretto
- Se $a[mid] < m$, allora, poiché l'array è ordinato, $a[i] \leq a[mid] < m$ per ogni $i \in \{left, \ldots, mid\}$
  - Quindi m è in $a[left..right]$ se e solo se è in $a[mid + 1..right]$
  - Questo intervallo ha dimensione $right - mid < k$ (perché $mid \geq left$), quindi per l'ipotesi induttiva la chiamata `RecursiveBinarySearch`$(a, mid + 1, right, m)$ termina e restituisce una risposta corretta, che viene restituita invariata

Dimostrazione (3/3)

- Se $a[mid] > m$, allora $a[i] \geq a[mid] > m$ per ogni $i \in \{mid, \ldots, right\}$
  - Quindi m è in $a[left..right]$ se e solo se è in $a[left..mid-1]$, un intervallo di dimensione $mid - left < k$ (perché $mid \leq right$)
  - Per l'ipotesi induttiva, la chiamata ricorsiva restituisce una risposta corretta

In tutti i casi la chiamata termina e restituisce una risposta corretta. □

- Applicando il Teorema 1 con $left = 1$ e $right = n$, otteniamo che `RecursiveBinarySearch`$(a, 1, n, m)$ risolve il problema

Tempo di esecuzione nel caso peggiore: l'equazione di ricorrenza

- Sia $S(n)$ il numero di passi eseguiti da `RecursiveBinarySearch` nel caso peggiore su un intervallo di $n = right - left + 1$ elementi (compresi i passi eseguiti da tutte le chiamate ricorsive)

- Il numero di passi eseguiti da una singola chiamata, **senza** contare i passi della chiamata ricorsiva, è:
  - intervallo vuoto: righe 2 e 3 $\Rightarrow$ 2 passi;
  - elemento trovato: righe 2, 5, 6, 7 $\Rightarrow$ 4 passi;
  - chiamata ricorsiva: righe 2, 5, 6, 9 e riga 10 (oppure 12) $\Rightarrow$ 5 passi.

Dimensione dei sottoproblemi

Sia $n = right - left + 1 \geq 1$. Poiché $mid - left = \lfloor (n-1)/2 \rfloor$:

- l'intervallo sinistro $a[left..mid-1]$ ha

$$mid - left = \left\lfloor \frac{n-1}{2} \right\rfloor$$

  elementi;
- l'intervallo destro $a[mid+1..right]$ ha

$$right - mid = (n-1) - \left\lfloor \frac{n-1}{2} \right\rfloor = \left\lceil \frac{n-1}{2} \right\rceil = \left\lfloor \frac{n}{2} \right\rfloor$$

  elementi.

Per esempio, per $n = 7$ entrambe le metà hanno 3 elementi; per $n = 8$ la metà sinistra ha 3 elementi e la metà destra ne ha 4.

L'equazione di ricorrenza

- Nel caso peggiore l'elemento non viene trovato prima che l'intervallo diventi vuoto, e la ricorsione prosegue sulla metà più grande, che ha $\lfloor n/2 \rfloor$ elementi (si verifica facilmente per induzione che S è non decrescente). Quindi,

$$S(n) = \begin{cases} 2 & \text{se } n = 0, \\ S\left(\lfloor n/2 \rfloor\right) + 5 & \text{se } n \geq 1. \end{cases}$$

- Questo caso peggiore si verifica, per esempio, quando $m > a[n]$: l'algoritmo prosegue sempre nella metà destra, finché l'intervallo diventa vuoto

- In generale, scriviamo $S(n) = S\left(\lfloor n/2 \rfloor\right) + c$, dove c è una costante (qui $c = 5$): un algoritmo ricorsivo che riduce il problema a **un** sottoproblema di dimensione **metà**, svolgendo una quantità **costante** di lavoro aggiuntivo

Soluzione per espansione (1/2)

Espandiamo l'equazione di ricorrenza, usando la proprietà $\lfloor \lfloor n/2^i \rfloor /2 \rfloor = \lfloor n/2^{i+1} \rfloor$:

$$\begin{aligned} S(n) &= S\big(\lfloor n/2 \rfloor\big) + 5 \\ &= S\big(\lfloor n/4 \rfloor\big) + 5 + 5 \\ &= S\big(\lfloor n/8 \rfloor\big) + 5 \cdot 3 \\ &\vdots \\ &= S\big(\lfloor n/2^j \rfloor\big) + 5\,j. \end{aligned}$$

L'espansione si ferma al primo j tale che $\lfloor n/2^j \rfloor = 0$, cioè $2^j > n$. Per $n \geq 1$, questo è

$$j = \lfloor \log_2 n \rfloor + 1,$$

perché $2^{\lfloor \log_2 n \rfloor} \leq n < 2^{\lfloor \log_2 n \rfloor + 1}$.

Soluzione per espansione (2/2)

- Quindi, per ogni $n \geq 1$,

$$S(n) = 5\big(\lfloor \log_2 n \rfloor + 1\big) + 2.$$

- Per esempio, $S(7) = 5 \cdot (2 + 1) + 2 = 17$: nella ricerca di $m = 10$ degli esempi, le tre chiamate su intervalli non vuoti eseguono 5 passi ciascuna e l'ultima chiamata (intervallo vuoto) ne esegue 2

- Poiché $\log_2 n - 1 < \lfloor \log_2 n \rfloor \leq \log_2 n$, abbiamo

$$\lim_{n\to\infty} \frac{S(n)}{\log_2 n} = \lim_{n\to\infty} \frac{5\lfloor \log_2 n \rfloor + 7}{\log_2 n} = 5 \in \mathbb{R}_{>0},$$

quindi

$$S(n) = \Theta(\log_2 n).$$

Soluzione con il Master theorem

Lo stesso risultato si ottiene immediatamente con il Master theorem, applicato a

$$S(n) = S(n/2) + c$$

(come di consueto, le parti intere inferiori non cambiano la soluzione asintotica).

- **Identificare** a, b, $f(n)$: $a = 1$, $b = 2$, $f(n) = c$
- **Calcolare** $n^{\log_b a} = n^{\log_2 1} = n^0 = 1$
- **Confrontare**: $f(n) = c = \Theta(1) = \Theta\left(n^{\log_2 1} \log^0 n\right)$
- **Verificare l'ipotesi**: siamo nel caso 2 del Master theorem con $k = 0$
- **Concludere**:

$$S(n) = \Theta\left(n^{\log_2 1} \log^{0+1} n\right) = \Theta(\log n) = \Theta(\log_2 n).$$

Numero di chiamate ricorsive

- Ogni chiamata su un intervallo non vuoto confronta m con esattamente un elemento $a[mid]$
- Quindi, il numero di chiamate su un intervallo non vuoto è il numero di elementi dell'array esaminati dall'algoritmo (esattamente come il numero di iterazioni del ciclo `while` dell'algoritmo iterativo)

Osservazione 1: numero di chiamate della ricerca binaria ricorsiva

Per $n \geq 1$, la chiamata `RecursiveBinarySearch`$(a, 1, n, m)$ *esegue al più*

$$\lfloor \log_2 n \rfloor + 1$$

chiamate su un intervallo non vuoto, e questo limite viene raggiunto (per esempio quando $m > a[n]$). Contando anche l'ultima chiamata su un intervallo vuoto, il numero totale di chiamate è al più $\lfloor \log_2 n \rfloor + 2$.

Dimostrazione (1/3)

Sia n_i la dimensione dell'intervallo della i-esima chiamata ($i = 1$ è la prima chiamata, quindi $n_1 = n$).

Per l'analisi dei sottoproblemi, $n_{i+1} \leq \lfloor n_i/2 \rfloor$. Per induzione su i,

$$n_i \leq \left\lfloor \frac{n}{2^{i-1}} \right\rfloor :$$

vale per $i = 1$, e se vale per i allora

$$n_{i+1} \leq \lfloor n_i/2 \rfloor \leq \left\lfloor \lfloor n/2^{i-1} \rfloor /2 \right\rfloor = \lfloor n/2^i \rfloor .$$

Dimostrazione (2/3)

Se la i-esima chiamata è su un intervallo non vuoto, allora $n_i \geq 1$, quindi $n/2^{i-1} \geq 1$, cioè

$$2^{i-1} \leq n \qquad \text{e} \qquad i - 1 \leq \log_2 n.$$

Poiché $i - 1$ è un intero, $i - 1 \leq \lfloor \log_2 n \rfloor$, cioè

$$i \leq \lfloor \log_2 n \rfloor + 1.$$

Quindi, al più $\lfloor \log_2 n \rfloor + 1$ chiamate sono su un intervallo non vuoto, e può seguirne al più un'altra (su un intervallo vuoto, che non effettua chiamate ricorsive).

Dimostrazione (3/3)

Se $m > a[n]$, l'algoritmo prosegue sempre nella metà destra, la cui dimensione è esattamente $\lfloor n_i/2 \rfloor$.

Quindi

$$n_i = \lfloor n/2^{i-1} \rfloor,$$

che è ≥ 1 per $i = \lfloor \log_2 n \rfloor + 1$: il limite viene raggiunto, ed è seguito da una chiamata su un intervallo vuoto. □

- Per esempio, per $n = 7$ si ha $\lfloor \log_2 7 \rfloor + 1 = 3$: nella ricerca di $m = 10$ ci sono 3 chiamate su intervalli non vuoti, più l'ultima chiamata sull'intervallo vuoto $left = 3$, $right = 2$
- Lo stesso ragionamento mostra che anche l'algoritmo iterativo esegue al più $\lfloor \log_2 n \rfloor + 1$ iterazioni del ciclo `while`

Tempo di esecuzione in funzione della dimensione dell'input

- L'input dell'algoritmo è costituito dall'array $a[1..n]$ e dal valore cercato m
- Sotto l'ipotesi che ogni valore usi un numero costante di bit, la dimensione dell'input soddisfa $|I| = \Theta(n)$, cioè esistono costanti $c_1, c_2 > 0$ tali che $c_1 n \leq |I| \leq c_2 n$ per ogni n sufficientemente grande
- Passando ai logaritmi,

$$\log_2 |I| - \log_2 c_2 \leq \log_2 n \leq \log_2 |I| - \log_2 c_1,$$

  quindi $\log_2 n = \Theta(\log_2 |I|)$ (le costanti additive sono trascurabili rispetto a $\log_2 |I|$)
- Poiché $S(n) = \Theta(\log_2 n)$, otteniamo $S(n) = \Theta(\log_2 |I|)$

Tempo di esecuzione nel caso peggiore di `RecursiveBinarySearch`

Se tutti i valori in ingresso usano un numero costante fissato di bit,
allora il tempo di esecuzione nel caso peggiore dell'algoritmo `RecursiveBinarySearch` è

$$T(|I|) = \Theta(\log |I|).$$

- Come la versione iterativa, l'algoritmo è logaritmico nella dimensione dell'input
- **Non** è necessario leggere l'intero input per risolvere il problema, perché l'array è ordinato

Spazio: ricerca binaria ricorsiva contro iterativa (1/2)

- L'algoritmo ricorsivo e quello iterativo eseguono gli stessi confronti e hanno lo stesso tempo di esecuzione nel caso peggiore. Differiscono per la **memoria** usata
- Ogni chiamata che non è ancora terminata mantiene in memoria una propria copia dei parametri e delle variabili locali (*left*, *right*, *m*, *mid*, un riferimento all'array e l'indirizzo di ritorno)
- Queste informazioni sono memorizzate nella **pila delle chiamate**, e occupano una quantità costante di memoria per chiamata

Spazio: ricerca binaria ricorsiva contro iterativa (2/2)

- In `RecursiveBinarySearch` nessuna chiamata termina prima che sia terminata la sua chiamata ricorsiva
- Quindi, quando è in esecuzione l'ultima chiamata, **tutte** le chiamate del cammino sono nella pila delle chiamate
- Per l'Osservazione 1, nel caso peggiore sono $\lfloor \log_2 n \rfloor + 2$, e la memoria aggiuntiva usata è

$$\Theta(\log n).$$

- L'algoritmo iterativo usa solo le variabili *left*, *right* e *mid*: la sua memoria aggiuntiva è

$$\Theta(1).$$

Ricorsione in coda

Definizione 1: ricorsione in coda

*Una chiamata ricorsiva è una **chiamata in coda** (tail call) se è l'ultima operazione eseguita dalla funzione chiamante, cioè il valore che restituisce viene restituito direttamente dal chiamante senza ulteriori calcoli. Una funzione le cui chiamate ricorsive sono tutte chiamate in coda si dice **ricorsiva in coda** (tail recursive).*

- `RecursiveBinarySearch` è ricorsiva in coda: nelle righe 10 e 12 il valore della chiamata ricorsiva viene restituito così com'è

Dalla ricorsione all'iterazione

- Una chiamata in coda si può sostituire aggiornando i parametri e tornando all'inizio della funzione:
  - la riga 10 diventa $left \leftarrow mid + 1$;
  - la riga 12 diventa $right \leftarrow mid - 1$;
  - il test della riga 2 diventa la condizione di un ciclo `while` «**while** $left \leq right$».

- Il risultato di questa trasformazione è **esattamente** l'algoritmo iterativo `BinarySearch`

- Ecco perché i due algoritmi producono la stessa successione di valori di *left*, *right* e *mid*

Eliminazione delle chiamate in coda

- Molti compilatori eseguono automaticamente questa trasformazione (**eliminazione delle chiamate in coda**, *tail-call elimination*), in modo che la versione ricorsiva usi solo $\Theta(1)$ di memoria aggiuntiva
- Per esempio, il compilatore g++ 15.2 elimina le chiamate ricorsive dell'implementazione C++ con l'opzione -O2, ma non con -O0
- Tuttavia, questo non è garantito dallo standard C++

Ricerca binaria ricorsiva contro iterativa

La ricerca binaria ricorsiva e quella iterativa
hanno lo stesso tempo di esecuzione nel caso peggiore
$\Theta(\log n)$,
ma la versione ricorsiva usa $\Theta(\log n)$ di memoria aggiuntiva
per la pila delle chiamate, mentre la versione iterativa usa
$\Theta(1)$ di memoria aggiuntiva.

Ogni algoritmo ricorsivo in coda
può essere trasformato in uno iterativo.

Riepilogo

Punti chiave sulla ricerca binaria ricorsiva:

- Una chiamata ricorsiva su (al più) metà dell'intervallo: l'albero delle chiamate è un cammino, con al più $\lfloor \log_2 n \rfloor + 2$ chiamate
- $S(n) = S(\lfloor n/2 \rfloor) + 5$, cioè $S(n) = 5(\lfloor \log_2 n \rfloor + 1) + 2 = \Theta(\log n)$ (caso 2 del Master theorem con $a = 1$)
- L'algoritmo è ricorsivo in coda: eliminando la ricorsione si ottiene l'algoritmo iterativo `BinarySearch`, con $\Theta(1)$ di memoria aggiuntiva invece di $\Theta(\log n)$

Il problema del sotto-array di somma massima

Introduzione: Il problema del sotto-array di somma massima

- L'ultimo problema del capitolo mostra il divide et impera al lavoro su un problema meno ovvio, in cui il passo *combina* è la parte interessante
- Tre algoritmi: forza bruta ($\Theta(n^2)$), divide et impera ($\Theta(n \log n)$) e un algoritmo iterativo ($\Theta(n)$)
- Un algoritmo ricorsivo che migliora la forza bruta non è necessariamente il migliore possibile

Motivazione: comprare e vendere un'azione

- Supponiamo di conoscere in anticipo il prezzo di un'azione per i giorni successivi
- Possiamo comprare un'unità di azione **una volta** e venderla **una volta**, in un giorno successivo. Vogliamo massimizzare il profitto (prezzo di vendita meno prezzo di acquisto)
- Nell'esempio della prossima slide, $p[d]$ è il prezzo al giorno d, e

$$a[d] = p[d] - p[d-1]$$

è la **variazione giornaliera** del prezzo

L'esempio è tratto dal libro di testo di T. H. Cormen, C. E. Leiserson, R. L. Rivest e C. Stein, *Introduction to Algorithms*, Sezione 4.1.

Prezzi e variazioni giornaliere

giorno d	0	1	2	3	4	5	6	7	8
prezzo $p[d]$	100	113	110	85	105	102	86	63	81
variazione $a[d]$		13	-3	-25	20	-3	-16	-23	18

giorno d	9	10	11	12	13	14	15	16
prezzo $p[d]$	101	94	106	101	79	94	90	97
variazione $a[d]$	20	-7	12	-5	-22	15	-4	7

Il profitto è una somma di variazioni giornaliere

- Se compriamo al giorno $l-1$ e vendiamo al giorno $r \geq l$, il profitto è

$$\begin{aligned} p[r] - p[l-1] &= \big(p[l] - p[l-1]\big) + \big(p[l+1] - p[l]\big) \\ &\quad + \cdots + \big(p[r] - p[r-1]\big) \\ &= a[l] + a[l+1] + \cdots + a[r], \end{aligned}$$

  cioè la somma delle variazioni giornaliere consecutive $a[l..r]$

- Quindi, massimizzare il profitto equivale a trovare la porzione consecutiva dell'array $a[1..16]$ con la **somma più grande**

Comprare basso e vendere alto non basta

- Si noti che **non** basta comprare al prezzo più basso o vendere al prezzo più alto: il prezzo più basso (63, giorno 7) viene dopo il prezzo più alto (113, giorno 1)

- In questo esempio la scelta migliore è comprare al giorno 7 (prezzo 63) e vendere al giorno 11 (prezzo 106), con profitto

$$106 - 63 = 43 = a[8] + a[9] + a[10] + a[11] = 18 + 20 - 7 + 12.$$

Il problema

Dato un array $a[1..n]$ di n numeri interi,
consideriamo il problema di trovare due indici $l \leq r$ tali che la somma

$$a[l] + a[l+1] + \cdots + a[r] = \sum_{k=l}^{r} a[k]$$

sia massima.

Il sotto-array $a[l..r]$ si chiama **sotto-array di somma massima** di a.

Osservazioni sul problema

- Un sotto-array è sempre **non vuoto** ($l \leq r$) e formato da elementi **consecutivi**
- Il problema è interessante solo se l'array contiene numeri negativi:
  - se tutti gli elementi sono non negativi, l'intero array $a[1..n]$ è un sotto-array di somma massima
  - se tutti gli elementi sono negativi, un sotto-array di somma massima è formato da un solo elemento, il più grande
- Un sotto-array di somma massima non è necessariamente unico: ne cerchiamo **uno**
- Il numero di sotto-array di $a[1..n]$ è il numero di coppie (l, r) con $1 \leq l \leq r \leq n$:

$$\sum_{l=1}^{n}(n-l+1) = n + (n-1) + \cdots + 1 = \frac{n(n+1)}{2}.$$

Un algoritmo di forza bruta

- L'algoritmo più semplice considera tutti gli $n(n+1)/2$ sotto-array $a[l..r]$ e tiene il migliore
- Per un l fissato, le somme di $a[l..r]$ per $r = l, l+1, \ldots, n$ si calcolano in modo incrementale: la somma di $a[l..r]$ è la somma di $a[l..r-1]$ più $a[r]$
- Nelle prossime slide, lo pseudocodice dell'algoritmo `BruteForceMaximumSubarray`

Algorithm: BruteForceMaximumSubarray (1/2)

Input : Un array $a[1..n]$ di n numeri interi
Output : Indici $bl \leq br$ di un sotto-array di somma massima
e la sua somma *best*

```
1 function BruteForceMaximumSubarray(a, n)
2 |   best := a[1]
3 |   bl := 1
4 |   br := 1
5 |   l := 1
6 |   while l ≤ n do
7 |   |   sum := 0
8 |   |   r := l
```

Algorithm: BruteForceMaximumSubarray (2/2)

```
function BruteForceMaximumSubarray(a, n) (continua)
    while l ≤ n (continua) do
9       while r ≤ n do
10          sum ← sum + a[r]                    // somma di a[l..r]
11          if sum > best then
12              best ← sum
13              bl ← l
14              br ← r
15          end
16          r ← r + 1
17      end
18      l ← l + 1
19  end
20  return (bl, br, best)
21 end
```

Esegui in Colab: Python · C++ · Java

Esempio

Consideriamo $a = [-2, 5, -1, 3]$. L'algoritmo parte con $best = a[1] = -2$ ed esamina i $4 \cdot 5/2 = 10$ sotto-array in quest'ordine:

l	r	sum	azione
1	1	-2	$sum \leq best = -2$
1	2	3	$sum > best \Rightarrow best = 3,\ bl = 1,\ br = 2$
1	3	2	
1	4	5	$sum > best \Rightarrow best = 5,\ bl = 1,\ br = 4$
2	2	5	$sum \leq best = 5$
2	3	4	
2	4	7	$sum > best \Rightarrow best = 7,\ bl = 2,\ br = 4$
3	3	-1	
3	4	2	
4	4	3	

L'algoritmo restituisce $(2, 4, 7)$: il sotto-array di somma massima è $a[2..4] = [5, -1, 3]$ con somma 7.

Tempo di esecuzione nel caso peggiore (1/4)

- Sia $S(n)$ il numero di passi eseguiti da `BruteForceMaximumSubarray` nel caso peggiore, dove n è la dimensione dell'array

- L'esecuzione delle righe 12–14 dipende dall'input. Definiamo una funzione $t(l, r) \in \{0, 1\}$, per $1 \leq l \leq r \leq n$, dove $t(l, r) = 1$ se le righe 12–14 vengono eseguite quando il ciclo interno esamina la coppia (l, r), e $t(l, r) = 0$ altrimenti

- Per un l fissato, il ciclo interno esegue $n - l + 1$ iterazioni, e la sua condizione viene verificata $n - l + 2$ volte. Quindi

$$\sum_{l=1}^{n}(n-l+1) = \frac{n(n+1)}{2}, \qquad \sum_{l=1}^{n}(n-l+2) = \frac{n(n+1)}{2} + n.$$

Tempo di esecuzione nel caso peggiore (2/4)

Il numero di volte che ciascuna riga di codice viene eseguita è:

righe	numero di esecuzioni (ciascuna)
2, 3, 4, 5	1
6	$n+1$
7, 8	n
9	$\frac{n(n+1)}{2} + n$
10, 11, 16	$\frac{n(n+1)}{2}$
12, 13, 14	$\sum_{l=1}^{n} \sum_{r=l}^{n} t(l,r)$
18	n
20	1

Tempo di esecuzione nel caso peggiore (3/4)

- Quindi, il numero totale di passi è

$$4 + (n+1) + 2n + \frac{n(n+1)}{2} + n + 3\,\frac{n(n+1)}{2} + n + 1 + 3\sum_{l=1}^{n}\sum_{r=l}^{n} t(l,r)$$

$$= 2n^2 + 7n + 6 + 3\sum_{l=1}^{n}\sum_{r=l}^{n} t(l,r).$$

- Nell'esempio ci sono 3 aggiornamenti e l'algoritmo esegue

$$2 \cdot 16 + 7 \cdot 4 + 6 + 3 \cdot 3 = 75$$

passi

Tempo di esecuzione nel caso peggiore (4/4)

- Poiché $0 \leq t(l, r) \leq 1$, la somma doppia è compresa tra 0 e $n(n+1)/2$. Quindi, per ogni input di dimensione n, il numero di passi è compreso tra

$$2n^2 + 7n + 6 \quad \text{e} \quad 2n^2 + 7n + 6 + \frac{3n(n+1)}{2} = \frac{7}{2}n^2 + \frac{17}{2}n + 6,$$

  e lo stesso vale per il numero di passi nel caso peggiore $S(n)$

- Entrambi i limiti sono $\Theta(n^2)$; per esempio,

$$\lim_{n\to\infty} \frac{2n^2 + 7n + 6}{n^2} = 2 \in \mathbb{R}_{>0},$$

$$\lim_{n\to\infty} \frac{\frac{7}{2}n^2 + \frac{17}{2}n + 6}{n^2} = \frac{7}{2} \in \mathbb{R}_{>0}.$$

  Quindi, $S(n) = \Theta(n^2)$

L'algoritmo di forza bruta è quadratico

- Si noti che, per **ogni** input (non solo nel caso peggiore), l'algoritmo esegue $\Theta(n^2)$ passi, poiché esamina sempre tutti gli $n(n+1)/2$ sotto-array
- Sotto l'ipotesi che ogni elemento usi un numero costante di bit, $|I| = \Theta(n)$, e quindi $S(n) = \Theta(|I|^2)$

Se tutti i valori in ingresso usano un numero costante fissato di bit,
allora il tempo di esecuzione nel caso peggiore dell'algoritmo `BruteForceMaximumSubarray` è

$$T(|I|) = \Theta(|I|^2).$$

Quindi, l'algoritmo di forza bruta è quadratico nella dimensione dell'input.

Un algoritmo divide et impera (1/2)

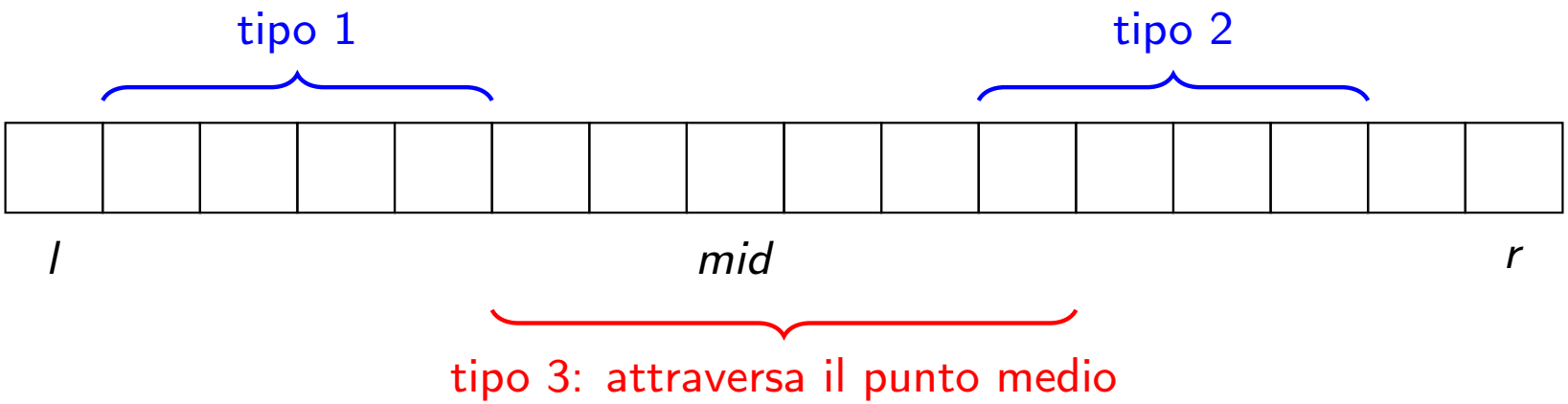
- Progettiamo ora un algoritmo ricorsivo basato sull'approccio **divide et impera**, come per il Merge Sort
- Risolviamo il problema su un sotto-array $a[l..r]$, e lo chiamiamo inizialmente con $l = 1$ e $r = n$
- **Divide**: si suddivide $a[l..r]$ in due metà $a[l..mid]$ e $a[mid + 1..r]$, dove

$$mid = \left\lfloor \frac{l + r}{2} \right\rfloor \quad \left(= l + \left\lfloor \frac{r - l}{2} \right\rfloor \right).$$

- Ogni sotto-array $a[i..j]$ di $a[l..r]$ (con $l \leq i \leq j \leq r$) si trova in una e una sola di queste tre situazioni:
  1. si trova interamente nella metà sinistra: $l \leq i \leq j \leq mid$;
  2. si trova interamente nella metà destra: $mid + 1 \leq i \leq j \leq r$;
  3. **attraversa il punto medio**: $l \leq i \leq mid < j \leq r$.

Un algoritmo divide et impera (2/2)

- **Impera**: un sotto-array di somma massima di tipo 1 e uno di tipo 2 si trovano **ricorsivamente**, poiché sono sotto-array di somma massima di $a[l..mid]$ e $a[mid + 1..r]$, rispettivamente
- **Combina**: un sotto-array di somma massima di tipo 3 si trova con una procedura non ricorsiva, `FindMaxCrossingSubarray`, descritta qui sotto. La risposta è il migliore dei tre sotto-array



Sotto-array di somma massima che attraversa il punto medio

- Un sotto-array che attraversa il punto medio è formato da due parti: $a[i..mid]$, che **termina** in mid, e $a[mid+1..j]$, che **inizia** in $mid+1$:

$$\sum_{k=i}^{j} a[k] = \sum_{k=i}^{mid} a[k] + \sum_{k=mid+1}^{j} a[k].$$

- Il primo termine dipende solo da i, il secondo solo da j: le due parti si possono scegliere **indipendentemente**
- Quindi, basta trovare il miglior i (scorrendo la metà sinistra da mid fino a l) e il miglior j (scorrendo la metà destra da $mid+1$ fino a r)
- Nelle prossime slide, lo pseudocodice dell'algoritmo `FindMaxCrossingSubarray` (dallo stesso libro di testo dell'esempio sull'azione)

Algorithm: FindMaxCrossingSubarray (1/2)

Input : Un array a e tre indici $l \leq mid < r$
Output : Indici $maxleft \leq mid < maxright$ di un sotto-array di somma massima che attraversa il punto medio, e la sua somma

```
1  function FindMaxCrossingSubarray(a, l, mid, r)
2      leftsum := −∞
3      sum := 0
4      for i := mid downto l do
5          sum ← sum + a[i]                    // somma di a[i..mid]
6          if sum > leftsum then
7              leftsum ← sum
8              maxleft ← i
9          end
10     end
```

Algorithm: FindMaxCrossingSubarray (2/2)

```
function FindMaxCrossingSubarray(a, l, mid, r) (continua)
11      rightsum := −∞
12      sum ← 0
13      for j := mid + 1 to r do
14          sum ← sum + a[j]          // somma di a[mid + 1..j]
15          if sum > rightsum then
16              rightsum ← sum
17              maxright ← j
18          end
19      end
20      return (maxleft, maxright, leftsum + rightsum)
21  end
```

Esegui in Colab: Python · C++ · Java

Lemma 1: sotto-array che attraversa il punto medio

Sia $l \leq mid < r$. `FindMaxCrossingSubarray`(a, l, mid, r) *restituisce* $(maxleft, maxright, s)$ *con* $l \leq maxleft \leq mid < maxright \leq r$ *e*

$$s = \sum_{k=maxleft}^{maxright} a[k]$$

$$= \max \left\{ \sum_{k=i}^{j} a[k] \ : \ l \leq i \leq mid < j \leq r \right\}.$$

Inoltre, esegue $\Theta(n)$ *passi, dove* $n = r - l + 1$.

Dimostrazione (1/3)

Ciclo sinistro. Alla fine dell'iterazione con indice i:

- la variabile *sum* è uguale a

$$L(i) = \sum_{k=i}^{mid} a[k];$$

- *leftsum* è il massimo tra $L(mid), L(mid-1), \ldots, L(i)$, raggiunto in *maxleft*

(per induzione sulle iterazioni: a ogni iterazione *sum* viene confrontato con il miglior valore trovato finora, che inizialmente è $-\infty$, quindi il primo confronto ha sempre successo).

Alla fine del ciclo,

$$leftsum = \max_{l \le i \le mid} L(i) = L(maxleft).$$

Dimostrazione (2/3)

Ciclo destro. Allo stesso modo, con $R(j) = \sum_{k=mid+1}^{j} a[k]$,

$$rightsum = \max_{mid+1 \le j \le r} R(j) = R(maxright).$$

Combinazione. Per ogni sotto-array che attraversa il punto medio, $\sum_{k=i}^{j} a[k] = L(i) + R(j)$. Poiché i e j si possono scegliere indipendentemente,

$$\begin{aligned} \max_{l \le i \le mid < j \le r} \big(L(i) + R(j)\big) &= \max_{l \le i \le mid} L(i) + \max_{mid < j \le r} R(j) \\ &= leftsum + rightsum, \end{aligned}$$

che è il valore restituito, raggiunto da $a[maxleft..maxright]$.

Dimostrazione (3/3)

Tempo di esecuzione.

- I due cicli eseguono $(mid - l + 1) + (r - mid) = r - l + 1 = n$ iterazioni in totale
- Ogni iterazione esegue un numero costante di passi (tra 3 e 5 righe, contando il test del ciclo)
- In più, un numero costante di passi fuori dai cicli

Quindi il numero di passi è compreso tra $c_1 n$ e $c_2 n$ per due costanti $c_1, c_2 > 0$, cioè $\Theta(n)$. □

- Si noti che il sotto-array che attraversa il punto medio contiene sempre sia $a[mid]$ sia $a[mid + 1]$, anche se sono negativi: i sotto-array di tipo 1 e 2 si occupano degli altri casi

L'algoritmo ricorsivo

Nelle prossime slide, lo pseudocodice dell'algoritmo `FindMaximumSubarray`.

Per risolvere il problema sull'intero array chiamiamo `FindMaximumSubarray`$(a, 1, n)$.

Algorithm: FindMaximumSubarray (1/2)

Input : Un array a e due indici $l \leq r$
Output : Indici $i \leq j$ di un sotto-array di somma massima di $a[l..r]$ e la sua somma

```
1 function FindMaximumSubarray(a, l, r)
2 |   if l = r then
3 |   |   return (l, r, a[l])      // Caso base: un elemento
4 |   end
5 |   mid := ⌊(l + r)/2⌋
6 |   (ll, lr, lsum) := FindMaximumSubarray(a, l, mid)
7 |   (rl, rr, rsum) := FindMaximumSubarray(a, mid + 1, r)
8 |   (cl, cr, csum) := FindMaxCrossingSubarray(a, l, mid, r)
```

Algorithm: FindMaximumSubarray (2/2)

```
function FindMaximumSubarray(a, l, r) (continua)
 9      if lsum ≥ rsum and lsum ≥ csum then
10          return (ll, lr, lsum)
11      end
12      if rsum ≥ lsum and rsum ≥ csum then
13          return (rl, rr, rsum)
14      end
15      return (cl, cr, csum)
16  end
```

Esegui in Colab: Python · C++ · Java

Come funziona `FindMaximumSubarray`

- Righe 2–3: se il sotto-array ha un solo elemento, il suo unico sotto-array è $a[l..l]$ (**caso base**)
- Riga 5: **divide** il sotto-array in $a[l..mid]$ e $a[mid + 1..r]$
- Righe 6–7: **impera**, cioè trova ricorsivamente un sotto-array di somma massima in ciascuna metà
- Riga 8: trova un sotto-array di somma massima che attraversa il punto medio
- Righe 9–15: **combina**, cioè restituisce il migliore dei tre (in caso di parità, si preferisce quello di sinistra, poi quello di destra)

- A differenza della ricerca binaria ricorsiva, ogni chiamata su almeno due elementi effettua **due** chiamate ricorsive: l'albero delle chiamate è un albero binario con n foglie (una per ogni elemento) e $n - 1$ nodi interni, cioè $2n - 1$ chiamate in totale, esattamente come per `RecursiveSum`

Esempio: l'esempio dell'azione

Consideriamo l'array delle variazioni giornaliere dell'esempio dell'azione:

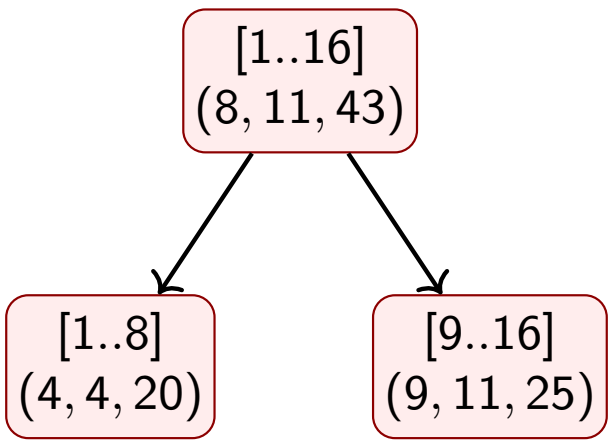
i	1	2	3	4	5	6	7	8
$a[i]$	13	−3	−25	20	−3	−16	−23	18

i	9	10	11	12	13	14	15	16
$a[i]$	20	−7	12	−5	−22	15	−4	7

Eseguiamo l'algoritmo divide et impera in quattro passi: l'albero delle chiamate, i risultati delle chiamate, i calcoli dei sotto-array che attraversano il punto medio per le due chiamate del livello 1, e quello per la prima chiamata.

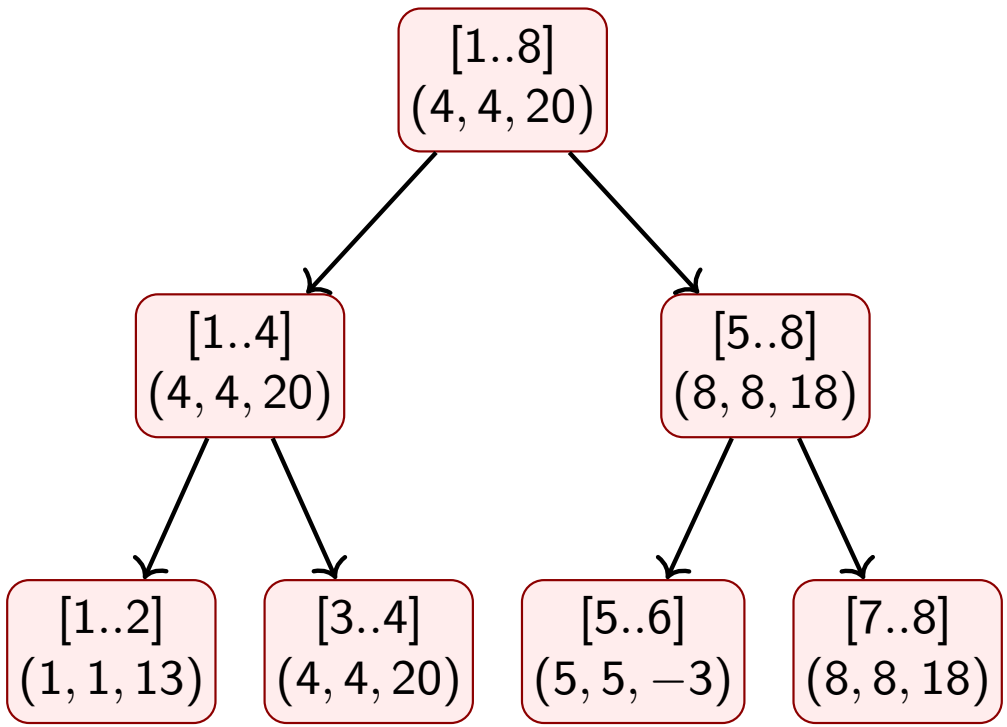
Esempio: passo 1, l'albero delle chiamate (1/3)

- Chiamiamo `FindMaximumSubarray`$(a, 1, 16)$
- Ogni nodo è una chiamata su $a[l..r]$, scritta come $[l..r]$, insieme alla terna (i, j, s) che restituisce
- Le 16 foglie (chiamate su un elemento $[k..k]$, che restituiscono $(k, k, a[k])$) non sono disegnate
- Prima la radice e i suoi due figli, poi i sottoalberi di [1..8] e di [9..16]



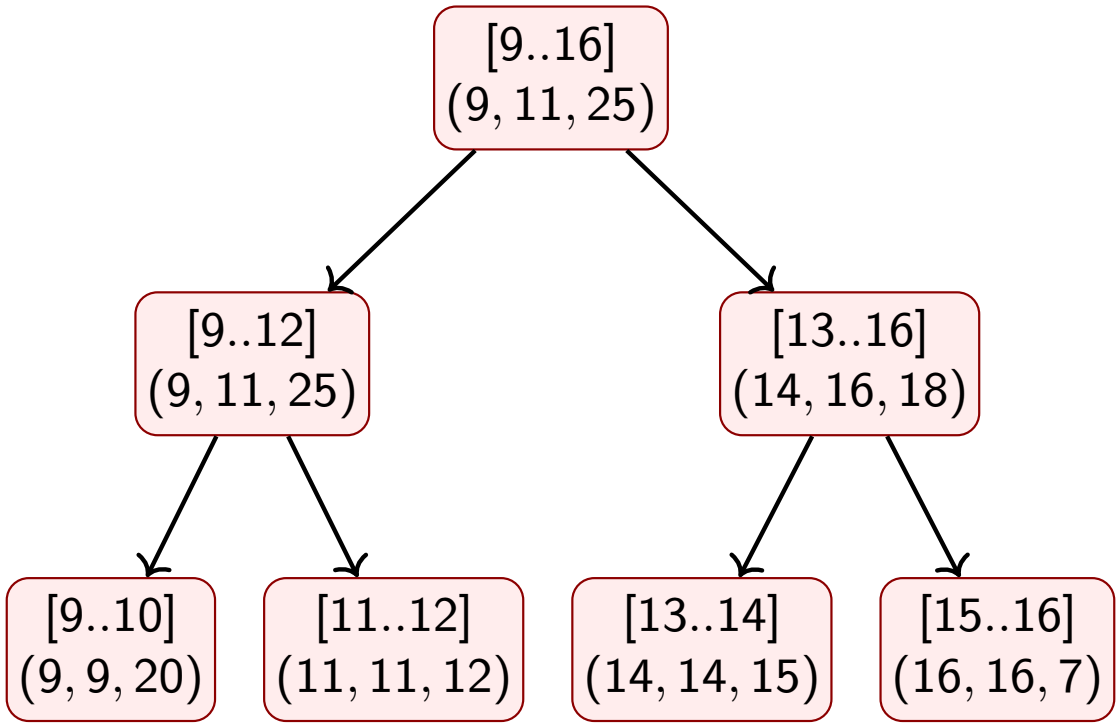
Esempio: passo 1, l'albero delle chiamate (2/3)

Il sottoalbero della chiamata [1..8]:



Esempio: passo 1, l'albero delle chiamate (3/3)

Il sottoalbero della chiamata [9..16]:



Esempio: passo 2, i risultati delle chiamate (1/2)

Per ogni chiamata su almeno due elementi: le terne restituite dalla chiamata ricorsiva sinistra, dalla chiamata ricorsiva destra e da `FindMaxCrossingSubarray`, e la terna restituita (la migliore delle tre).

chiamata	*mid*	sinistra	destra	attraversante	restituita
[1..16]	8	(4, 4, 20)	(9, 11, 25)	(8, 11, 43)	(8, 11, 43)
[1..8]	4	(4, 4, 20)	(8, 8, 18)	(4, 5, 17)	(4, 4, 20)
[9..16]	12	(9, 11, 25)	(14, 16, 18)	(9, 16, 16)	(9, 11, 25)
[1..4]	2	(1, 1, 13)	(4, 4, 20)	(1, 4, 5)	(4, 4, 20)
[5..8]	6	(5, 5, −3)	(8, 8, 18)	(6, 8, −21)	(8, 8, 18)
[9..12]	10	(9, 9, 20)	(11, 11, 12)	(9, 11, 25)	(9, 11, 25)
[13..16]	14	(14, 14, 15)	(16, 16, 7)	(14, 16, 18)	(14, 16, 18)

Esempio: passo 2, i risultati delle chiamate (2/2)

Le chiamate su due elementi:

chiamata	*mid*	sinistra	destra	attraversante	restituita
$[1..2]$	1	$(1, 1, 13)$	$(2, 2, -3)$	$(1, 2, 10)$	$(1, 1, 13)$
$[3..4]$	3	$(3, 3, -25)$	$(4, 4, 20)$	$(3, 4, -5)$	$(4, 4, 20)$
$[5..6]$	5	$(5, 5, -3)$	$(6, 6, -16)$	$(5, 6, -19)$	$(5, 5, -3)$
$[7..8]$	7	$(7, 7, -23)$	$(8, 8, 18)$	$(7, 8, -5)$	$(8, 8, 18)$
$[9..10]$	9	$(9, 9, 20)$	$(10, 10, -7)$	$(9, 10, 13)$	$(9, 9, 20)$
$[11..12]$	11	$(11, 11, 12)$	$(12, 12, -5)$	$(11, 12, 7)$	$(11, 11, 12)$
$[13..14]$	13	$(13, 13, -22)$	$(14, 14, 15)$	$(13, 14, -7)$	$(14, 14, 15)$
$[15..16]$	15	$(15, 15, -4)$	$(16, 16, 7)$	$(15, 16, 3)$	$(16, 16, 7)$

Esempio: passo 3, il punto medio nelle chiamate del livello 1 (1/2)

Chiamata [1..8], $mid = 4$:

i	$a[i]$	sum	$leftsum$	$maxleft$
4	20	20	20	4
3	−25	−5	20	4
2	−3	−8	20	4
1	13	5	20	4

j	$a[j]$	sum	$rightsum$	$maxright$
5	−3	−3	−3	5
6	−16	−19	−3	5
7	−23	−42	−3	5
8	18	−24	−3	5

Il sotto-array che attraversa il punto medio è $a[4..5]$ con somma $20 + (-3) = 17$.

Esempio: passo 3, il punto medio nelle chiamate del livello 1 (2/2)

Chiamata [9..16], $mid = 12$:

i	$a[i]$	sum	$leftsum$	$maxleft$
12	-5	-5	-5	12
11	12	7	7	11
10	-7	0	7	11
9	20	20	20	9

j	$a[j]$	sum	$rightsum$	$maxright$
13	-22	-22	-22	13
14	15	-7	-7	14
15	-4	-11	-7	14
16	7	-4	-4	16

Il sotto-array che attraversa il punto medio è $a[9..16]$ con somma $20 + (-4) = 16$.

Esempio: passo 4, il punto medio nella prima chiamata (1/3)

Chiamata [1..16], $mid = 8$, ciclo sinistro:

i	$a[i]$	*sum*	*leftsum*	*maxleft*
8	18	18	18	8
7	−23	−5	18	8
6	−16	−21	18	8
5	−3	−24	18	8
4	20	−4	18	8
3	−25	−29	18	8
2	−3	−32	18	8
1	13	−19	18	8

Esempio: passo 4, il punto medio nella prima chiamata (2/3)

Chiamata [1..16], $mid = 8$, ciclo destro:

j	$a[j]$	*sum*	*rightsum*	*maxright*
9	20	20	20	9
10	-7	13	20	9
11	12	25	25	11
12	-5	20	25	11
13	-22	-2	25	11
14	15	13	25	11
15	-4	9	25	11
16	7	16	25	11

Il sotto-array che attraversa il punto medio è $a[8..11]$ con somma $18 + 25 = 43$.

Esempio: passo 4, il punto medio nella prima chiamata (3/3)

- Nella prima chiamata, la metà sinistra dà 20, la metà destra dà 25 e il sotto-array che attraversa il punto medio dà 43
- L'algoritmo restituisce $(8, 11, 43)$, cioè si compra al giorno 7 e si vende al giorno 11, con profitto 43
- Si noti che il sotto-array ottimo viene trovato solo all'ultimo passo, da `FindMaxCrossingSubarray`: non si trova né nella metà sinistra né nella metà destra

Teorema 2: correttezza di FindMaximumSubarray

Per ogni $l \leq r$, `FindMaximumSubarray`(a, l, r) *restituisce* (i, j, s) *con* $l \leq i \leq j \leq r$ *e*

$$s = \sum_{k=i}^{j} a[k] = \max \left\{ \sum_{k=i'}^{j'} a[k] \ : \ l \leq i' \leq j' \leq r \right\}.$$

- Come per tutti gli algoritmi ricorsivi del capitolo, la dimostrazione è per induzione forte sul numero di elementi del sotto-array

Dimostrazione (1/2)

Procediamo per induzione forte sul numero di elementi $n = r - l + 1 \geq 1$.

Caso base $n = 1$. Allora $l = r$, l'unico sotto-array è $a[l..l]$, e l'algoritmo restituisce $(l, l, a[l])$, il che è corretto.

Passo induttivo. Sia $n \geq 2$ e supponiamo (ipotesi induttiva) che l'enunciato valga per ogni sotto-array con $n' < n$ elementi. Poiché $l < r$, si ha $l \leq mid < r$, e le due metà $a[l..mid]$ e $a[mid + 1..r]$ hanno $\lceil n/2 \rceil$ e $\lfloor n/2 \rfloor$ elementi, entrambi almeno 1 e minori di n.

Dimostrazione (2/2)

Per l'ipotesi induttiva:

- *lsum* è la somma massima di un sotto-array di tipo 1 (contenuto in $a[l..mid]$), raggiunta da $a[ll..lr]$;
- *rsum* è la somma massima di un sotto-array di tipo 2 (contenuto in $a[mid+1..r]$), raggiunta da $a[rl..rr]$.

Per il Lemma 1, *csum* è la somma massima di un sotto-array di tipo 3 (che attraversa il punto medio), raggiunta da $a[cl..cr]$.

Ogni sotto-array di $a[l..r]$ è di tipo 1, 2 o 3. Quindi la somma massima di un sotto-array di $a[l..r]$ è

$$\max\{lsum, rsum, csum\},$$

e le righe 9–15 restituiscono esattamente una terna che raggiunge questo massimo. □

Tempo di esecuzione nel caso peggiore: l'equazione di ricorrenza

- Sia $S(n)$ il numero di passi eseguiti da `FindMaximumSubarray` nel caso peggiore su un sotto-array di $n = r - l + 1$ elementi
- Se $n = 1$, vengono eseguite solo le righe 2–3: $S(1) = c_1$, una costante
- Se $n \geq 2$:
  - le righe 2, 5 e 9–15 eseguono un numero costante di passi;
  - le chiamate ricorsive delle righe 6 e 7 eseguono $S(\lceil n/2 \rceil)$ e $S(\lfloor n/2 \rfloor)$ passi;
  - la riga 8 esegue $\Theta(n)$ passi (Lemma 1).
- Quindi,

$$S(n) = \begin{cases} c_1 & \text{se } n = 1, \\ S\big(\lceil n/2 \rceil\big) + S\big(\lfloor n/2 \rfloor\big) + \Theta(n) & \text{se } n \geq 2. \end{cases}$$

La stessa equazione di ricorrenza del Merge Sort

- Ignorando le parti intere inferiori e superiori (che non cambiano la soluzione asintotica), otteniamo la stessa equazione di ricorrenza del Merge Sort:

$$S(n) = 2\,S(n/2) + \Theta(n).$$

- Si noti che il tempo di esecuzione non dipende dai valori nell'array, ma solo da n: ogni input di dimensione n è un caso peggiore

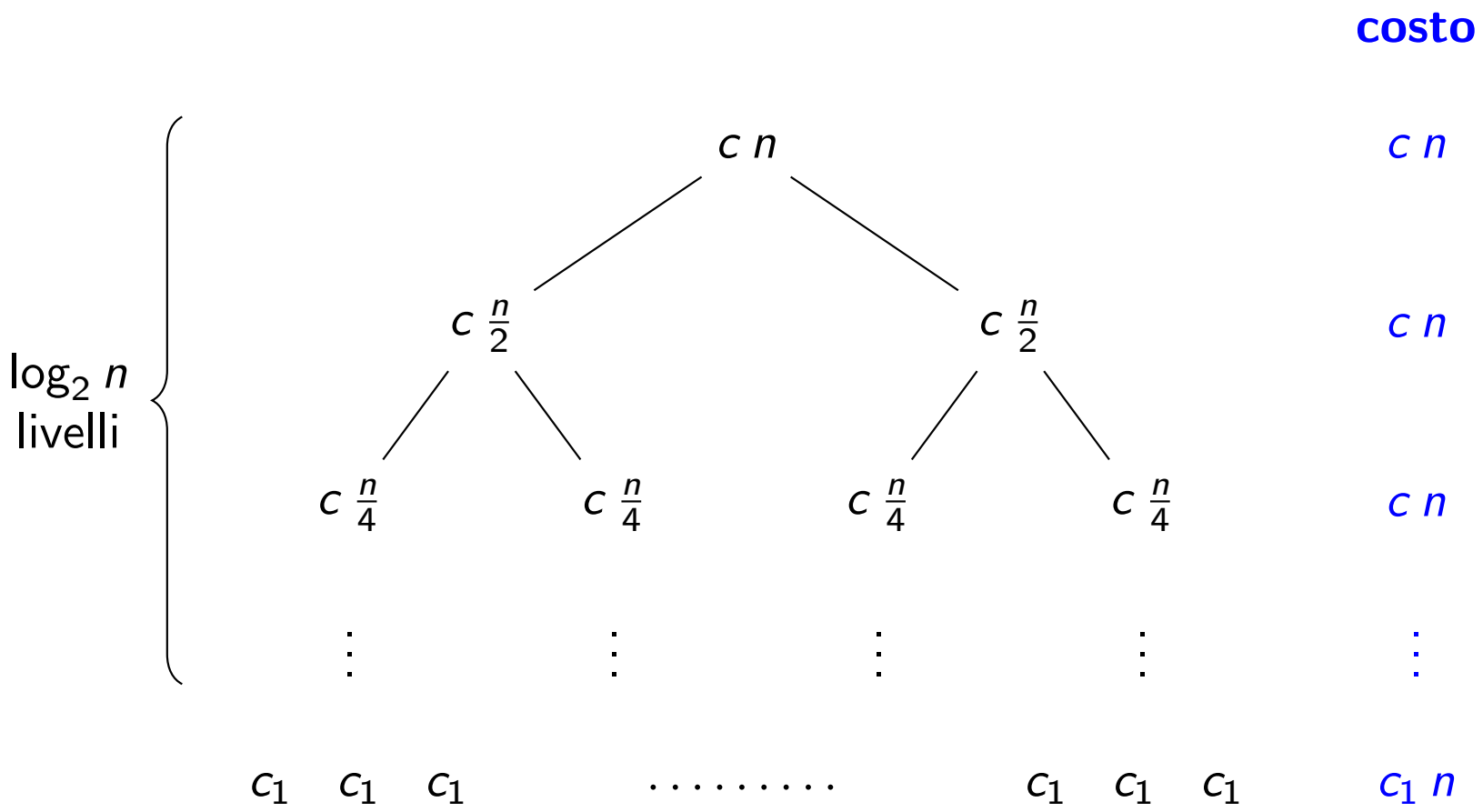
Soluzione con il Master theorem:

- **Identificare** a, b, $f(n)$: $a = 2$, $b = 2$, $f(n) = \Theta(n)$
- **Calcolare** $n^{\log_b a} = n^{\log_2 2} = n$
- **Confrontare**: $f(n) = \Theta(n) = \Theta\left(n^{\log_2 2} \log^0 n\right)$
- **Verificare l'ipotesi**: siamo nel caso 2 del Master theorem con $k = 0$
- **Concludere**: $S(n) = \Theta\left(n^{\log_2 2} \log^{0+1} n\right) = \Theta(n \log n)$

Soluzione con l'albero di ricorsione

- Sia $n = 2^k$ una potenza di 2, e sia $c\,n$ il costo della parte non ricorsiva di una chiamata su $n \geq 2$ elementi
- Nell'albero di ricorsione, il livello h (per $h = 0, 1, \ldots, k-1$) contiene 2^h chiamate su $n/2^h$ elementi ciascuna, con costo totale
$$2^h \cdot c\,\frac{n}{2^h} = c\,n.$$
- L'ultimo livello ($h = k = \log_2 n$) contiene le n foglie, con costo totale $c_1 n$
- L'albero è nella prossima slide

L'albero di ricorsione di `FindMaximumSubarray`



Sommando sui livelli

- Sommando su tutti i livelli:

$$S(n) = \underbrace{c\,n + c\,n + \cdots + c\,n}_{\log_2 n \text{ volte}} + c_1 n$$
$$= c\,n \log_2 n + c_1 n = \Theta(n \log n).$$

- L'albero di ricorsione spiega anche il risultato: ci sono $\log_2 n$ livelli e ogni livello costa $\Theta(n)$, perché i calcoli dei sotto-array che attraversano il punto medio, per le chiamate di un livello, scorrono tutti insieme l'intero array una sola volta

- Sotto l'ipotesi che ogni elemento usi un numero costante di bit, $|I| = \Theta(n)$, e anche $\log n = \Theta(\log |I|)$ (come mostrato per la ricerca binaria ricorsiva). Quindi

$$S(n) = \Theta(n \log n) = \Theta(|I| \log |I|).$$

Tempo di esecuzione nel caso peggiore di `FindMaximumSubarray`

Se tutti i valori in ingresso usano un numero costante fissato di bit,
allora il tempo di esecuzione nel caso peggiore dell'algoritmo `FindMaximumSubarray` è

$$T(|I|) = \Theta(|I| \log |I|).$$

Quindi, l'algoritmo divide et impera è asintoticamente più veloce
dell'algoritmo di forza bruta,
il cui tempo di esecuzione nel caso peggiore è $\Theta(|I|^2)$.

Osservazione: un algoritmo iterativo lineare

- L'algoritmo divide et impera è molto meglio dell'algoritmo di forza bruta
- Tuttavia, il problema del sotto-array di somma massima si può risolvere ancora più velocemente, con un semplice algoritmo **iterativo** dovuto a J. Kadane, che scorre l'array una **sola** volta
- **Idea**: per ogni posizione i, sia $M(i)$ la somma massima di un sotto-array che **termina in** posizione i
- Un sotto-array che termina in i è o il solo $a[i]$, oppure un sotto-array che termina in $i-1$ esteso con $a[i]$. Quindi,

$$M(1) = a[1], \qquad M(i) = a[i] + \max\{0, M(i-1)\} \quad (i \geq 2).$$

Ripartire o estendere

- A parole: se il miglior sotto-array che termina in $i-1$ ha somma negativa, conviene **ripartire** da $a[i]$; altrimenti, conviene **estenderlo**
- La risposta al problema è il massimo tra $M(1), \ldots, M(n)$
- Nelle prossime slide, lo pseudocodice dell'algoritmo `Kadane`. La variabile *cur* memorizza $M(i)$ e *cl* il primo indice del sotto-array corrispondente

Algorithm: Kadane (1/2)

Input : Un array $a[1..n]$ di n numeri interi
Output : Indici $bl \leq br$ di un sotto-array di somma massima
e la sua somma *best*

```
1  function Kadane(a, n)
2      best := a[1]; bl := 1; br := 1
3      cur := a[1]; cl := 1
4      i := 2
5      while i ≤ n do
6          if cur < 0 then
7              cur ← a[i]; cl ← i                    // ripartire
8          else
9              cur ← cur + a[i]                      // estendere
10         end
```

Algorithm: Kadane (2/2)

```
function Kadane(a, n) (continua)
    while i ≤ n (continua) do
11      if cur > best then
12          best ← cur; bl ← cl; br ← i
13      end
14      i ← i + 1
15  end
16  return (bl, br, best)
17 end
```

Esegui in Colab: Python · C++ · Java

Lemma 2: invariante di ciclo di Kadane

All'inizio di ogni iterazione del ciclo `while` *(e quando il ciclo termina), con indice corrente* i*:*

1. $cur = \sum_{k=cl}^{i-1} a[k]$ *è la somma massima di un sotto-array che* ***termina in*** *posizione* $i-1$*;*
2. $best = \sum_{k=bl}^{br} a[k]$ *è la somma massima di un sotto-array di* $a[1..i-1]$*.*

Dimostrazione (1/3)

Inizializzazione ($i = 2$). L'unico sotto-array che termina in 1 è $a[1..1]$, ed è anche l'unico sotto-array di $a[1..1]$: $cur = best = a[1]$ con $cl = bl = br = 1$.

Mantenimento. Supponiamo che l'invariante valga all'inizio dell'iterazione con indice i.

- Un sotto-array che termina in i è o $a[i..i]$, con somma $a[i]$, oppure $a[k..i]$ con $k < i$, la cui somma è

$$\sum_{h=k}^{i-1} a[h] + a[i] \leq cur + a[i]$$

(per la parte 1), con uguaglianza per $k = cl$

Dimostrazione (2/3)

- Quindi la somma massima di un sotto-array che termina in i è

$$\max\{a[i], cur + a[i]\},$$

che è $a[i]$ se $cur < 0$ e $cur + a[i]$ altrimenti: è esattamente ciò che calcolano le righe 6–9. Quindi la parte 1 vale per $i + 1$

- Un sotto-array di $a[1..i]$ o è un sotto-array di $a[1..i-1]$ (somma migliore: *best*, per la parte 2) oppure termina in i (somma migliore: il nuovo *cur*). Le righe 11–12 mantengono il maggiore dei due, quindi la parte 2 vale per $i + 1$

Dimostrazione (3/3)

Terminazione. Il ciclo termina con $i = n + 1$: per la parte 2, *best* è la somma massima di un sotto-array di $a[1..n]$, raggiunta da $a[bl..br]$. □

Tempo di esecuzione.

- Il ciclo `while` esegue $n - 1$ iterazioni, ciascuna con un numero costante di passi
- Quindi $S(n) = \Theta(n)$ e, se ogni elemento usa un numero costante di bit, $T(|I|) = \Theta(|I|)$
- Poiché qualunque algoritmo deve leggere tutti gli elementi dell'array, nessun algoritmo può essere asintoticamente più veloce

Esempio (1/2)

Esecuzione di `Kadane` sull'esempio dell'azione $a = [13, -3, -25, 20, -3, -16, -23, 18, 20, -7, 12, -5, -22, 15, -4, 7]$. Ogni riga riporta i valori alla fine dell'iterazione con indice i (riga $i = 1$: inizializzazione).

i	$a[i]$	azione	cur	cl	$best$	(bl, br)
1	13	inizializzazione	13	1	13	$(1, 1)$
2	-3	estensione	10	1	13	$(1, 1)$
3	-25	estensione	-15	1	13	$(1, 1)$
4	20	ripartenza ($cur < 0$)	20	4	**20**	$(4, 4)$
5	-3	estensione	17	4	20	$(4, 4)$
6	-16	estensione	1	4	20	$(4, 4)$
7	-23	estensione	-22	4	20	$(4, 4)$
8	18	ripartenza ($cur < 0$)	18	8	20	$(4, 4)$

Esempio (2/2)

i	$a[i]$	azione	cur	cl	$best$	(bl, br)
9	20	estensione	38	8	**38**	$(8, 9)$
10	-7	estensione	31	8	38	$(8, 9)$
11	12	estensione	43	8	**43**	$(8, 11)$
12	-5	estensione	38	8	43	$(8, 11)$
13	-22	estensione	16	8	43	$(8, 11)$
14	15	estensione	31	8	43	$(8, 11)$
15	-4	estensione	27	8	43	$(8, 11)$
16	7	estensione	34	8	43	$(8, 11)$

L'algoritmo restituisce $(8, 11, 43)$, lo stesso sotto-array di somma massima trovato dall'algoritmo divide et impera, dopo un'unica scansione dell'array.

Riepilogo

Punti chiave sul problema del sotto-array di somma massima:

- algoritmo di forza bruta: $\Theta(n^2)$
- algoritmo divide et impera (ricorsivo): $\Theta(n \log n)$, con la stessa equazione di ricorrenza del Merge Sort
- algoritmo di Kadane (iterativo): $\Theta(n)$

La lezione:

- Un algoritmo ricorsivo divide et impera non è necessariamente il migliore possibile: qui, un semplice algoritmo iterativo che sfrutta la struttura del problema ($M(i)$ dipende solo da $M(i-1)$) è asintoticamente più veloce
- Torneremo su questa lezione nel capitolo «Algoritmi ricorsivi contro iterativi»

Riepilogo del capitolo

Introduzione: Riepilogo del capitolo

I quattro problemi del capitolo coprono le tre forme tipiche di un algoritmo ricorsivo sugli array:

- due chiamate sulle metà con lavoro di combinazione **costante** (somma)
- due chiamate sulle metà con lavoro di combinazione **lineare** (Merge Sort, sotto-array di somma massima)
- **una** chiamata su una metà (ricerca binaria)

Gli algoritmi ricorsivi del capitolo (1/2)

Le equazioni di ricorrenza:

algoritmo	ricorrenza
`RecursiveSum`	$S(n) = 2S(n/2) + c$
`RecursiveSumLinear`	$S(n) = S(n-1) + 3$
`MergeSort`	$S(n) = 2S(n/2) + \Theta(n)$
`RecursiveBinarySearch`	$S(n) = S(\lfloor n/2 \rfloor) + 5$
`FindMaximumSubarray`	$S(n) = 2S(n/2) + \Theta(n)$

Gli algoritmi ricorsivi del capitolo (2/2)

Numero di passi e memoria aggiuntiva:

algoritmo	numero di passi	memoria extra
`RecursiveSum`	$7n - 5 = \Theta(n)$	$\Theta(\log_2 n)$
`RecursiveSumLinear`	$3n - 1 = \Theta(n)$	$\Theta(n)$
`MergeSort`	$\Theta(n \log_2 n)$	$\Theta(n)$
`RecursiveBinarySearch`	$5(\lfloor \log_2 n \rfloor + 1) + 2 = \Theta(\log_2 n)$	$\Theta(\log_2 n)$
`FindMaximumSubarray`	$\Theta(n \log_2 n)$	$\Theta(\log_2 n)$

Riepilogo (1/2)

Correttezza:

- sempre per induzione forte sulla dimensione del sottoproblema
- le parti non ricorsive (`Merge`, `FindMaxCrossingSubarray`, `Kadane`) con invarianti di ciclo

Numero di passi:

- un'equazione di ricorrenza, risolta con l'albero di ricorsione, per espansione o con il Master theorem
- caso 1 per la somma (le foglie dominano), caso 2 per il Merge Sort, il sotto-array di somma massima e la ricerca binaria (tutti i livelli hanno lo stesso costo)

Riepilogo (2/2)

Quando conviene la ricorsione?

- La ricorsione dà un guadagno asintotico solo quando il lavoro del passo di combinazione, sommato su un livello, è piccolo rispetto a quello che fa un algoritmo iterativo (il Merge Sort rispetto agli algoritmi di ordinamento $\Theta(n^2)$)
- Per la somma e per la ricerca non dà alcun guadagno in tempo e costa memoria
- Per il sotto-array di somma massima il miglior algoritmo è iterativo

Memoria:

- la pila delle chiamate costa memoria proporzionale alla profondità della ricorsione